AGRO — AI Farming Companion

Expanded Project Documentation

NYLP AI Hackathon 2026 • Team HSR • Individual project

Project at a glance

AGRO is an AI farming companion that helps farmers make crop-care decisions using their farm records, measurements, and local weather.

This document contains the expanded project README and the complete NYLP AI Hackathon 2026 submission-form details, including workflows, features, implementation, data handling, safety limitations, setup guidance, evaluation notes, and the current local-development status.

Team lead and individual contributor: U.G.Harshani Sandunika Ranasingha
Contact: harshanisadunika99@gmail.com • WhatsApp: 0727713839, 0769956712

Prepared as a supporting document. The project is not publicly deployed; public demo, repository, and screenshot links still require participant input.

Project README

AGRO — AI Farming Companion

AGRO is a web application that helps farmers organize farm and crop information, ask crop-specific questions, and review weather and recorded measurements in one place. Its AI assistant can use a farmer's saved crop records and, when relevant, look up recorded measurements or a location-based forecast before answering.

AGRO is an international hackathon MVP: the core product is designed to be adaptable to different farming contexts, but recommendations have not been validated for every region, crop, or climate. Treat AI responses as guidance to review with qualified local agricultural experts.

NYLP AI Hackathon 2026 submission summary

Project title: AGRO — AI Farming Companion

Tagline: An AI farming companion that helps farmers make crop-care decisions using their farm records, measurements, and local weather.

Category: AI Agent (primary); related categories: Chatbots and Data Analytics. Domain: AgriTech.

Team information

- Team lead: U.G.Harshani Sandunika Ranasingha
- Email and registration email: harshanisadunika99@gmail.com
- WhatsApp: 0727713839, 0769956712
- Team name: HSR
- Participation type: Individual
- Team size: 1
- Team member and role: U.G.Harshani Sandunika Ranasingha — individual contributor

Project summary

AGRO is a web-based AI farming companion designed to help farmers worldwide make more informed crop-care decisions. It connects an AI chat assistant with farm and crop profiles, saved observations, recorded measurements, and nearby weather forecasts. When relevant, it retrieves those records, explains which information it used, and offers cautious next steps without presenting estimates as measurements or claiming a certain diagnosis.

Problem: Farmers may not have timely access to practical, crop-specific guidance that considers their farm conditions and saved observations. General AI answers can be unreliable when they guess at weather, soil measurements, or diagnoses.

Solution: AGRO combines an AI chat assistant with a farmer's crop profiles, saved observations, recorded numeric measurements, and a nearby weather forecast. It can retrieve those records when relevant, explain what information it used, and offer cautious next steps.

Highlights

- Personalized farm profiles with location, land size, irrigation, coordinates, and language preference.
- Worldwide location search and coordinate selection using the Google Maps picker.
- Crop management with variety, planting date, growth stage, soil information, and notes.
- Ollama-powered crop chat using farm and crop context, saved conversations, and the local text model.
- A crop journal for observations, weather, soil tests, growth, watering, fertilizer use, expenses, and income.
- Measurement trends based on saved numeric data, without estimating missing values.
- AI data tools that retrieve recent crop records, measurement trends, and local weather; the interface shows which data sources were checked.
- Current weather, forecasts, and historical climate information for farm coordinates. Weather is based on a nearby grid, not a direct field measurement.
- Provisional crop-planning suggestions informed by the available farm profile, crop history, and forecast.

- Local crop-photo review with a safety disclaimer that describes visible evidence; photos cannot confirm disease or measure soil nutrients or pH.
- Document questions for PDF, DOCX, TXT, CSV, and Markdown files.
- Local voice transcription with Whisper via Transformers.js and FFmpeg; recordings are not sent to OpenAI in local mode.
- English, Sinhala, or Tamil language preferences and browser speech playback for answers.

Innovation and responsible use

AGRO connects conversational AI to a farmer's own crop journal and numeric observations, and can retrieve a local forecast as a separate data source. Its read-only tools and visible action summaries help distinguish saved measurements and forecast data from AI interpretation. It emphasizes uncertainty: it does not infer soil nutrients from ordinary photos or present disease suggestions as confirmed diagnoses.

AGRO is intended as a flexible international farming companion. Recommendations should be adapted to local crops, climate, and agricultural guidance. AI responses are provisional and are not a substitute for advice from a qualified local agricultural expert.

Submission technology summary

React 18 and Vite 6 (frontend); Node.js and Express 4 (backend); MongoDB and Mongoose (database); Ollama llama3.2:latest (local text agent and crop planning) and qwen2.5vl:3b (local vision); Transformers.js Whisper Tiny and FFmpeg (local voice transcription); PDF and DOCX text extraction; optional OpenAI API provider; Google Maps Platform APIs; Open-Meteo; JavaScript, HTML, and CSS.

Development status

Working hackathon MVP for local development and testing. Authenticated API smoke tests have verified crop chat and measurement-tool calls, crop planning, crop-photo analysis with a safety disclaimer, PDF and DOCX text uploads, and local voice transcription. On 2026-10-18, all 16 backend tests passed, the frontend production build succeeded, and the backend dependency audit reported zero vulnerabilities. The frontend, backend health endpoint, and Ollama responded with HTTP 200; the configured Ollama text model (llama3.2:latest) and vision model (qwen2.5vl:3b) are installed. This is local-development verification, not a public deployment. No public deployment, repository, screenshot, or demo-video link has been provided.

This summary is included as supporting documentation for the NYLP AI Hackathon 2026 project submission.

The problem

Farmers need practical guidance that considers their crops, farm conditions, and observations over time. Generic advice can miss this context, while unsupported estimates about weather, soil, or crop problems can lead to poor decisions.

The solution

AGRO combines crop and farm records with AI chat and external weather data. Farmers can save what they observe and measure, then ask questions grounded in that information. The assistant has limited, read-only tools for retrieving crop history, recorded numeric measurements, and local weather. Its actions are shown in the chat so users can see which data sources were checked.

Features

- Farmer accounts: Register and log in to a private account; manage a worldwide farm address, map coordinates, land size, irrigation, and language preferences.
- Google Maps location picker: Search for an address or select a point on an interactive map; AGRO saves the place address and coordinates for weather and farm context.
- Crop profiles: Save crop name, variety, location, planting date, growth stage, soil details, irrigation, and notes.
- Crop-specific AI chat: Ask questions with crop and farm context, continue saved conversations, and see summaries of data lookups.
- AI crop planning: Request crop suggestions informed by available farmer profile details, crop history, and forecast data.
- Crop journal: Record observations for weather, soil, growth, crop problems, watering, fertilizer, expenses,

income, and other notes.

- Measurement charts: Store numeric values and visualize recorded trends. AGRO does not fill in missing measurements.
- Weather and climate: Retrieve current conditions and a five-day forecast, plus a one-year historical climate view, for entered coordinates through Open-Meteo.
- AI data tools: Retrieve recent crop observations, recorded numeric trends, and a five-day local forecast when relevant. Tools are read-only and limited per response.
- Photo and file questions: Review JPEG, PNG, or WebP crop photos with a local vision model; extract text locally from supported PDF/DOCX reports and TXT, CSV, or Markdown files.
- Voice questions: Record audio in the browser and transcribe it locally with Whisper; OpenAI transcription remains available in cloud mode.
- Language preferences: English, Sinhala, or Tamil.
- Spoken answers: Play assistant responses with the browser's speech synthesis support.
- Private attachments: Uploaded files are associated with the authenticated farmer and crop and stored in MongoDB.

A closer look at the farmer workflow

AGRO is organized around keeping a farmer's own records available when a question is asked. The first useful step is to create an account and save the farm information the farmer is comfortable providing. A farm address or map coordinates can support location-aware weather; land size, irrigation, and language preference add useful background. The location picker can search an address and select a point on a map when the Google Maps key has been configured. Without that key, address and manual coordinate fields remain available, but map selection and address lookup are disabled.

The farmer can then create one or more crop profiles. A profile can hold the crop name, variety, planting date, growth stage, location, soil details, irrigation information, and notes. These fields are not treated as a complete agronomic assessment. Rather, they give the assistant and farmer a shared reference for identifying which crop a question concerns and which facts have actually been saved. Accurate and current records make the surrounding context more useful; missing information remains missing and should be supplied by the farmer or checked with a local expert.

During the growing season, the crop journal provides a place to add observations and practical records. Examples include a visible change in the crop, watering or fertilizer activity, weather notes, a soil test result, growth progress, expenses, or income. Numeric entries can be reviewed as measurement trends. The distinction between a number actually entered by the farmer and an estimate generated by a model is important: AGRO's charts visualize saved values, and its measurement lookup does not fill gaps or manufacture a trend from absent readings.

When asking a crop question, the farmer can use the crop chat and select the relevant crop. The service assembles available farm and crop details, the farmer's language preference, recent conversation context, and recent saved observations. If a question needs a source that is not already in that context, the text model can request a limited read-only lookup for crop history, an actual measurement trend, or local weather. The chat displays a summary of data-source actions so the farmer can distinguish information retrieved for the answer from the model's explanation. The farmer can continue saved conversations rather than re-entering all context for every question.

Weather is useful for local context but must be interpreted carefully. When coordinates are available, the app can request current conditions and a five-day forecast, and display historical climate information. Open-Meteo weather is associated with a nearby grid point, not a sensor installed in the farmer's field. A forecast can inform planning, but it should not be presented as a measured amount of rainfall or as proof of the conditions at a particular plant.

Crop planning is a separate request from ordinary conversation. The available farm profile, recent crop records, and forecast can be supplied to help produce provisional suggestions. A farmer should compare those suggestions with local growing seasons, seed availability, water access, market context, local rules, and advice from agricultural officers. AGRO does not guarantee yields, profitability, or suitability for a particular farm.

Questions can include different media. Supported crop-photo formats are JPEG, PNG, and WebP. A configured vision model can describe visible evidence, but a photograph alone cannot establish a certain disease, pest, soil pH, or nutrient level. PDF and DOCX documents are converted to text locally before being used for analysis; TXT, CSV, and Markdown files are read as text. Scanned-image PDFs need OCR and are not supported by the documented local extraction workflow. Browser-recorded voice questions are decoded with FFmpeg and

transcribed using local Whisper Tiny in Ollama mode; the transcript, rather than the audio recording, is then used for the assistant request. The first local transcription may require downloading the model from Hugging Face.

Context, provider choices, and data boundaries

The implementation separates the user interface, authenticated API, persisted records, and AI or external-data providers. React and Vite provide the browser application. The Express API authenticates requests and communicates with MongoDB through Mongoose. The AI provider is selected through backend configuration: Ollama is the documented local-development default, while an OpenAI provider remains available as an explicit alternative. Weather comes from Open-Meteo, not from either AI provider. The map and address-search integration uses Google Maps Platform when a restricted browser API key is configured.

In the local setup, llama3.2:latest handles text chat, crop planning, and the read-only data-tool cycle. qwen2.5vl:3b handles image review. Transformers.js runs the Whisper Tiny transcription model locally, with FFmpeg used for audio decoding. This split makes it clearer which component performs each task: transcription converts speech to text, the text model answers questions, the vision model reviews image content, and the weather provider supplies forecasts. Local inference avoids sending these AI requests to OpenAI, but the first model downloads need network access and local inference still depends on the computer's available resources.

For crop chat, the backend includes the farmer's selected language preference, profile and crop details, recent crop-specific conversation history, and recent observations when available. The assistant can request get_crop_history for up to 12 recent observations, get_measurement_trend for up to 30 recorded values for a supported metric, and get_local_weather when farm coordinates are present. The tool cycle is bounded to at most four rounds and three lookups per round. These tools read data; they do not edit a crop record or create a measurement. If coordinates or a requested measurement are unavailable, the system cannot truthfully return a field value for it.

File processing has its own boundary. The documented per-file upload limit is 10 MB. In local mode, PDF/DOCX text extraction and audio transcription occur locally before content is passed into the configured text-model workflow. Plain text, CSV, and Markdown are included as text. In the optional cloud-provider configuration, requests to OpenAI are subject to that provider's service and account terms; users must not assume that data stays on their device. Before using real farm or personal information, review the chosen provider, obtain appropriate consent, and configure secrets securely. Uploaded files and crop records are persisted in MongoDB under the owning account.

These boundaries are intended to improve transparency, not to imply that generated recommendations are verified. Models may misunderstand a question, overlook a relevant record, or produce plausible but inaccurate advice. Users should inspect the sources summarized in chat, correct outdated records, and seek qualified local agricultural advice for consequential decisions.

Technology stack

Area • Technologies

Frontend • React 18, Vite 6, JavaScript, HTML, CSS

Backend • Node.js, Express 4

Database • MongoDB, Mongoose 8

AI • Optional OpenAI API; local Ollama text and vision models

Speech transcription • Transformers.js Whisper Tiny and FFmpeg locally; optional OpenAI whisper-1 cloud transcription

Weather • Open-Meteo forecast and historical archive APIs

Location search and map picker • Google Maps Platform Maps JavaScript API and Geocoding API (optional key required)

Authentication • Password hashing with bcryptjs; signed JSON Web Tokens

Upload handling • Multer memory storage; 10 MB per-file limit

Requirements

- Node.js 20.16+ or 22.3+ and npm

- Either a local MongoDB server or a MongoDB Atlas cluster
- Either an OpenAI API key with available credits or Ollama installed locally with the required text model; vision and transcription models are needed for all local media features

Weather views use Open-Meteo and do not require an API key. Local Ollama mode runs chat, crop planning, image analysis, and document review without OpenAI credits. Local inference requires adequate computer memory and processing capacity.

Run locally

Run commands from the project root unless a step says otherwise. You can use a local MongoDB server or MongoDB Atlas. With Atlas, there is no local database service to start; the backend connects over the internet.

1. Configure the backend

In PowerShell:

```
cd backend
npm.cmd install
Copy-Item .env.example .env
```

Option A: MongoDB Atlas (no local MongoDB service)

- Create a MongoDB Atlas account and create a cluster.
- In Atlas, create a database user and save its username and password securely. This is separate from your Atlas website login.
- In Network Access, add your current IP address so your computer can connect. Avoid allowing access from anywhere (0.0.0.0/0) except temporarily for testing; remove broad access afterward.
- Choose Connect → Drivers → Node.js and copy the connection string.
- In backend/.env, set MONGODB_URI to that URI. Replace <db_password> with the database user's password, URL-encoding special characters in the password if necessary, and add the database name agro_ai_farming_companion before any query parameters.

Example format (replace all placeholders; do not copy this as-is):

```
MONGODB_URI=mongodb+srv://<database-user>:<encoded-password>@<cluster-host>/
agro_ai_farming_companion?retryWrites=true&w=majority
```

Option B: Local MongoDB

Keep MongoDB running locally and use:

```
MONGODB_URI=mongodb://127.0.0.1:27017/agro_ai_farming_companion
```

Complete backend configuration

Open backend/.env and set the database URI from Option A or B, plus the other values:

```
PORT=5000
MONGODB_URI=your-mongodb-connection-uri
JWT_SECRET=replace-with-a-private-random-secret-at-least-32-characters-long
OPENAI_API_KEY=
OPENAI_MODEL=gpt-4o-mini
AI_PROVIDER=ollama
AI_BASE_URL=http://127.0.0.1:11434/v1
AI_MODEL=llama3.2:latest
AI_VISION_MODEL=qwen2.5vl:3b
AI_TRANSCRIPTION_MODEL=onnx-community/whisper-tiny
CLIENT_ORIGIN=http://localhost:5173,http://127.0.0.1:5173
```

Generate a unique, private JWT_SECRET of at least 32 characters. Keep .env out of source control; never put

your Atlas URI, database password, or API keys in frontend code, screenshots, or chat. This example selects local Ollama. OpenAI remains optional and requires setting AI_PROVIDER=openai, OPENAI_API_KEY, and (optionally) OPENAI_MODEL.

No-cost local AI with Ollama

OpenAI API usage requires available credits. To use a local model instead:

- Install Ollama for Windows from ollama.com/download (https://ollama.com/download).
- Open PowerShell and download the local text and vision models:

```
ollama pull llama3.2:latest
ollama pull qwen2.5vl:3b
```

- Make sure Ollama is running, then set these values in backend/.env:

```
AI_PROVIDER=ollama
AI_BASE_URL=http://127.0.0.1:11434/v1
AI_MODEL=llama3.2:latest
AI_VISION_MODEL=qwen2.5vl:3b
AI_TRANSCRIPTION_MODEL=onnx-community/whisper-tiny
```

OPENAI_API_KEY is not needed in local mode. Keep MONGODB_URI and JWT_SECRET configured as usual.

- Restart the backend after saving .env. Keep Ollama running while using AGRO.

Local mode supports crop chat, AGRO's crop-data tools, crop recommendations, crop-photo analysis, PDF/DOCX text extraction, and local voice transcription. The text and vision models are served by Ollama. The Whisper speech model is downloaded from Hugging Face by Transformers.js on first use and cached locally; audio is decoded locally with FFmpeg and is not sent to OpenAI. The first transcription therefore needs an internet connection to download the model. Scanned-image PDFs require OCR and are not supported; ordinary text-based PDFs and DOCX files are supported. Plain text, CSV, and Markdown attachments are included as text. Local model speed depends on your computer.

Start the backend from the backend directory:

```
npm.cmd run dev
```

The API starts at http://localhost:5000. Check that it is available at http://localhost:5000/api/health.

2. Start the frontend

Open a second terminal:

```
cd frontend
npm.cmd install
npm.cmd run dev
```

Open the local URL Vite prints, usually http://localhost:5173.

Configure Google Maps location search and picker

The farm and crop location map requires a Google Maps Platform API key. In Google Cloud Console, enable billing and the Maps JavaScript API and Geocoding API for a project, create an API key, and restrict it to the APIs and website referrers you use (for local development, http://localhost:5173/* and http://127.0.0.1:5173/*). Browser keys are visible to users by design; restrict them and never reuse a server-side secret key.

Create frontend/.env.local from the example and set the browser key:

```
VITE_API_URL=http://localhost:5000/api
VITE_GOOGLE_MAPS_API_KEY=your-restricted-browser-key
```

Restart Vite after changing environment variables. Without a Maps key, the address and manual coordinate fields remain available, but map selection and address lookup are disabled.

The frontend uses http://localhost:5000/api by default. Set VITE_API_URL in frontend/.env.local only when your backend uses another URL.

3. Try the app

- Register an account and choose a language preference.
- Add farm details and coordinates if you want location-based weather.
- Create a crop profile.
- Add crop observations and measured values to build a journal and trends.
- Ask a question in the crop chat; attach a supported photo or document, or record a voice question.
- Ask for crop-planning suggestions and review the farm information and forecast used.
- Refresh the page and confirm your profile, crops, chat, observations, and attachments persist.

AI methodology and implementation

The backend constructs the assistant request from a system prompt, recent crop-specific conversation history, the farmer's language preference, farm and crop profile fields, and recent saved observations. Responses use the configured OpenAI model or local Ollama model.

For crop chat, the model may request one or more of the following read-only tools:

- get_crop_history returns up to 12 recent observations for the selected crop.
- get_measurement_trend returns up to 30 actual recorded numeric values for a supported metric, such as soil pH, soil moisture, rainfall, or temperature.
- get_local_weather retrieves current conditions and a five-day forecast from Open-Meteo when farm coordinates are saved.

Tool use is bounded to a maximum of four tool rounds and three lookups per round. The app records a summary of tool actions alongside the assistant message. Crop recommendations use a separate AI request with available profile details, recent crop records, and a forecast when coordinates are available.

Images are sent to the configured vision model for visual review. PDF and DOCX text is extracted locally before analysis; TXT, CSV, and Markdown content is included as text. In Ollama mode, audio recordings are transcribed locally before the transcript is sent to the assistant. File uploads are limited to 10 MB.

The assistant is instructed to distinguish farmer-reported observations from inference, avoid inventing measurements or prices, avoid claiming certainty from limited evidence, and recommend qualified local advice for serious or uncertain crop problems. These safeguards reduce risk but cannot guarantee that generated advice is correct.

In local mode, text chat uses Ollama's OpenAI-compatible endpoint, while crop-photo analysis uses Ollama's native /api/chat endpoint for image input.

System workflow

The local Ollama text model handles crop chat, crop-data tools, and crop planning. Image analysis uses a separate Ollama vision model; PDF and DOCX uploads are converted to text before analysis. Voice recordings are transcribed locally before their transcript is sent to the text model. The text and vision models must be pulled with Ollama; the Whisper model downloads from Hugging Face on first transcription.

AI models and APIs

Provider • Model/API • Use

OpenAI • gpt-4o-mini by default; configurable with OPENAI_MODEL • Optional cloud text chat, crop-planning recommendations, image and document analysis

Ollama • llama3.2:latest by default; configurable with AI_MODEL • Local text chat, crop-data tools, and crop-planning recommendations

Ollama • qwen2.5vl:3b by default; configurable with AI_VISION_MODEL • Local crop-photo visual review

Transformers.js • onnx-community/whisper-tiny by default; configurable with AI_TRANSCRIPTION_MODEL • Local CPU audio transcription

OpenAI • whisper-1 • Optional cloud audio transcription in OpenAI mode

Open-Meteo • Forecast API • Current weather and forecast; not an AI model

Open-Meteo • Historical Archive API • Historical climate data; not an AI model

OpenAI usage is billed according to the account and model. A valid key must also have available API access/ credits. Do not commit or publish the key.

Team contributions

- U.G.Harshani Sandunika Ranasingha — individual contributor: Full-stack project implementation, including the React/Vite interface, Express/MongoDB backend, crop and weather features, AI assistant integration, and tests.

Please review this contribution statement and adjust it if any part does not accurately reflect the work completed.

Tests

Run backend tests from the backend directory:

```
npm.cmd test
```

The backend test suite has 16 tests covering direct agent answers, crop-history lookup, recorded measurement retrieval, weather lookup and missing-coordinate handling, safe stopping when the tool-round limit is reached, local PDF/DOCX extraction, image-analysis request handling, and safety disclaimers. Authenticated API smoke tests have also verified local text chat, crop planning, crop-photo review, PDF/DOCX uploads, and speech transcription.

Latest local verification (2026-10-18): all 16 backend tests passed, the frontend production build succeeded, and npm.cmd audit reported zero vulnerabilities. The frontend, backend health endpoint, and Ollama responded with HTTP 200. The configured local text model (llama3.2:latest) and vision model (qwen2.5vl:3b) were installed. These checks describe this development environment; they do not certify a public deployment.

Security, safety, and limitations

- This is a hackathon MVP, not a production-ready agricultural diagnostic or decision system.
- Generated responses can be inaccurate. Confirm important crop, disease, fertilizer, and pesticide decisions with qualified local agricultural experts and official product-label instructions.
- Weather and historical climate data refer to a nearby grid location; they are not measurements taken in the field.
- An ordinary photo cannot reliably determine soil pH or nutrient levels. Use actual measurements or a soil-test report.
- Recommendations can be incomplete when location, season, climate, soil, water, or other local details are missing. The app does not provide authoritative market prices or guaranteed yields.
- Agricultural advice has not been validated for every region or crop. Review and localize recommendations before relying on them in a new area.
- Uploaded files and crop records are stored in MongoDB under the owning account. Only share real farmer data with appropriate consent.
- Before public deployment, add and test production controls such as deployment secrets management, upload scanning, backups, privacy/consent documentation, monitoring, and broader security and integration testing.

NYLP AI Hackathon 2026 Submission Form

NYLP AI Hackathon 2026 – Project Submission Form

Form cover

Form logo: [Insert NYLP AI Hackathon 2026 logo here]

Welcome to the project submission stage of the NYLP AI Hackathon 2026!

It's time to showcase what you've built. Submit your AI-powered product, demonstrate how it works, and share your repository and project documentation.

To keep the submission process simple, please share links to your resources using Google Drive, GitHub, or other accessible platforms. Make sure the judges have permission to access all submitted links.

Please complete all required fields before submitting your project.

Required fields are marked with an asterisk (*).

Participant & Team Information

Team Lead's Full Name *

U.G.Harshani Sandunika Ranasingha

Team Lead's Email Address *

harshanisadunika99@gmail.com

WhatsApp Number *

0727713839, 0769956712

Registration Email Address *

harshanisadunika99@gmail.com

Team Name *

HSR

Participation Type *

individual

Number of Team Members *

1

Team Members' Names and Roles *

U.G.Harshani Sandunika Ranasingha — individual contributor

Project Details

Project Title *

AGRO — AI Farming Companion

Project Tagline *

An AI farming companion that helps farmers make crop-care decisions using their farm records, measurements, and local weather.

Project Category *

AI Agent (primary); related categories: Chatbots and Data Analytics. Domain: AgriTech.

Problem Statement *

Farmers may not have timely access to practical, crop-specific guidance that considers their farm conditions and saved observations. General AI answers can also be unreliable when they guess at weather, soil measurements, or diagnoses.

Proposed Solution *

AGRO combines an AI chat assistant with a farmer's crop profiles, saved observations, recorded numeric measurements, and a nearby weather forecast. It can retrieve those records when relevant, explain what information it used, and offer cautious next steps without treating estimates as measurements or claiming a certain diagnosis.

Project Description *

AGRO is a web-based AI farming companion designed to help farmers worldwide make more informed crop-care decisions. In the current local setup, AGRO uses Ollama's llama3.2:latest model for text chat, crop-data tools, and crop planning, with qwen2.5vl:3b for crop-photo review and local Whisper transcription for voice questions. These local features do not require an OpenAI API key. Its features include:

- Personalized farm profiles: Save farm location, land size, irrigation, coordinates, and language preference.
- Worldwide location selection: Search for a farm or crop address and select its coordinates using the Google Maps picker.
- Crop management: Create crop profiles with variety, planting date, growth stage, soil information, and notes.
- Ollama-powered crop chat: Ask questions with farm and crop context, access saved conversations, and get responses from the locally running Ollama model.
- Crop journal: Record farmer observations, weather, soil tests, growth, watering, fertilizer use, expenses, and income.
- Measurement trends: Review saved numeric measurements without estimating missing values.
- AI data tools: Let the Ollama-powered assistant retrieve recent crop records, measurement trends, and local weather when relevant; the interface shows which data sources it checked.
- Weather information: View current conditions, forecasts, and historical climate information for saved farm coordinates. Weather describes a nearby grid and is not a direct field measurement.
- Ollama-powered crop planning: Get provisional crop suggestions informed by the available farm profile, crop history, and forecast.
- Local photo review: Send a crop photo to Ollama's vision model for a description of visible evidence; a photo cannot confirm disease or measure soil nutrients or pH.
- Document questions: Extract text locally from supported PDF and DOCX documents; include TXT, CSV, and Markdown files as text.
- Local voice transcription: Transcribe recorded questions on-device with Whisper via Transformers.js and FFmpeg; recordings are not sent to OpenAI.
- Multilingual experience: Use English, Sinhala, or Tamil language preferences.
- Spoken responses: Listen to AGRO's answers using browser speech playback.

AGRO is intended as a flexible international farming companion; recommendations should be adapted to local crops, climate, and agricultural guidance. AI responses are provisional and are not a substitute for advice from a qualified local agricultural expert.

Detailed Product Workflow

AGRO is designed to make a farmer's own information available at the point where they ask for help. A typical workflow begins with the farmer registering and entering a farm profile. The profile can capture a farm address, coordinates, land size, irrigation details, and language preference. If the Google Maps Platform key is enabled and restricted for the frontend, the farmer can search an address or select a point on the map. The coordinates make nearby weather requests possible. Without a configured Maps key, manual address and coordinate fields remain available, while map selection and address lookup are disabled.

The farmer can create a crop profile to give the application a specific subject for future questions. The profile can contain a crop name and variety, planting date, growth stage, location, soil information, irrigation information, and

notes. The profile is intended as a record of farmer-provided context; it is not an automated soil survey or a guarantee that all agronomic variables are known. If the farmer does not know a value or has not entered it, the assistant should not represent it as a recorded fact.

As the season progresses, the crop journal can preserve observations and activities such as changes in growth, crop problems, soil tests, weather observations, watering, fertilizer use, expenses, and income. Numeric measurements can be viewed over time as trends. The product uses the recorded values as its evidence rather than filling blank dates with generated numbers. That distinction matters to farmers making decisions: a real test result, an observation, a model interpretation, and a forecast are different kinds of information and should not be confused.

For a question, the farmer can open crop chat and ask in their preferred language. The backend can include the selected crop and farm details, saved observations, and recent crop-specific conversation history in the model request. When additional evidence would help, the text model may ask the backend to retrieve recent crop history, a supported measurement trend, or a local forecast. The lookups are read-only and their action summaries are shown with the answer. This is intended to help the farmer see what AGRO checked, rather than presenting an answer as though it came from an unexplained or all-knowing source.

Weather context is requested separately from the language model. Where farm coordinates are saved, Open-Meteo can provide current conditions, a five-day forecast, and historical climate information. The reported weather is for a nearby grid location; it is not a direct reading from a sensor in the farmer's field. The forecast may help frame a question about planning, but it should not be mistaken for observed rainfall, soil moisture, or the condition of an individual plot.

Crop planning can provide provisional options based on the available farm profile, crop history, and forecast. Such suggestions need local review because crop suitability can depend on soil testing, season, water supply, seed availability, local pest pressure, market conditions, and government guidance. The project does not promise yields or profits. A farmer should use local agricultural expertise before making high-impact decisions based on AI-generated suggestions.

The product also supports questions that begin with media. A supported crop photo can be sent to the configured vision model for a description of visible evidence. A photo cannot confirm a disease or reliably measure soil pH or nutrients. PDF and DOCX files are converted to text locally before analysis, while TXT, CSV, and Markdown can be included as text. The documented PDF extraction workflow handles text-based PDFs and does not provide OCR for scanned image-only pages. In the local configuration, browser-recorded voice questions are decoded with FFmpeg and transcribed with Whisper Tiny through Transformers.js; the audio is processed locally and the resulting transcript is used for the question. The first transcription may download the model from Hugging Face. In local Ollama mode, these features do not require an OpenAI API key.

AI and Data-Handling Explanation

The project separates text chat, image review, speech transcription, weather retrieval, and persistence rather than treating them as one undifferentiated AI operation. Ollama llama3.2:latest is used for local text chat, crop planning, and bounded data-tool requests. Ollama qwen2.5vl:3b is used for local visual review. Transformers.js runs onnx-community/whisper-tiny for local speech transcription, and FFmpeg decodes audio. Open-Meteo supplies weather information and is not an AI model. The optional OpenAI configuration can be selected by an installation that explicitly configures it; in that case, its applicable service, privacy, and billing terms should be reviewed.

For a crop-chat request, the backend can provide the model with the farmer's language preference, selected crop and farm profile, recent saved observations, and relevant conversation context. Three read-only lookups are available: get_crop_history returns up to 12 recent observations; get_measurement_trend can return up to 30 actually recorded numeric values for supported metrics; and get_local_weather uses saved coordinates to request nearby weather. The lookup cycle is limited to four tool rounds with at most three lookups in each round. Those limits help keep the workflow bounded. A lookup does not write changes to a farmer's crop record, and absent coordinates or measurements cannot produce a valid real-world observation.

AGRO's safety approach is based on communicating the limits of the available evidence. It aims to distinguish farmer-reported facts from generated interpretation, numeric measurements from forecasts, and visible image descriptions from confirmed diagnosis. It should not infer soil nutrients from an ordinary crop photograph, invent missing readings, or claim a disease is certain from limited evidence. These measures reduce the risk of overconfident presentation but cannot guarantee that every response will be complete or correct. Farmers should verify consequential recommendations with qualified local experts and official product instructions.

The application uses React 18 and Vite 6 in the browser, Node.js and Express 4 for the API, and MongoDB with Mongoose for persisted accounts and farm data. The local-development architecture includes authenticated API access, model configuration in the backend, and separate integrations for weather and optional map services. Uploaded files and crop records are associated with the owning account in MongoDB. The documented per-file upload limit is 10 MB. Users should avoid placing private keys in frontend code or sharing real farmer data without appropriate consent. Local model inference avoids sending the AI request to OpenAI, but models may need to be downloaded initially and the computer must have sufficient resources to run them.

How Judges Can Assess the Product

An accessible local demonstration can show the relationship between records and answers without implying that a public service is deployed. A useful walkthrough would begin with a farm profile and one crop profile, then show a saved journal observation and a numeric measurement. The presenter can ask a question about the selected crop and point out the data-source actions displayed alongside the answer. If test coordinates and network access are available, the presenter can demonstrate a weather lookup while explaining that it is from a nearby grid rather than a field sensor. Crop planning can then be reviewed as a provisional suggestion, not a guarantee.

Media features can be demonstrated separately. A crop image can be used to show the vision model's description and its safety disclaimer. A text-based PDF or DOCX can demonstrate local text extraction, while a short voice question can demonstrate local transcription after any first-use model download. For each example, judges can check whether the application explains what source was used and whether the response avoids presenting unsupported conclusions as measured facts. A meaningful evaluation should also consider missing context: if coordinates, a soil result, or a crop record is absent, a reliable system should disclose that limitation rather than invent a value.

The implementation's current verification is limited to the local development setup. Authenticated API smoke tests have covered crop chat and measurement-tool calls, crop planning, crop-photo analysis with a safety disclaimer, PDF and DOCX uploads, and local voice transcription. On 2026-10-18, all 16 backend tests passed, the frontend production build succeeded, and the backend dependency audit reported zero vulnerabilities. The frontend, backend health endpoint, and configured Ollama models responded successfully in that environment. These checks do not establish public deployment, production readiness, validation across all crops and regions, or expert approval of agricultural recommendations.

The final deliverable remains a working hackathon MVP with a clear distinction between locally verified features and submission items that still need participant action. No public product, repository, screenshots folder link, or demo-video link has been provided in this workspace. Those should be added and permissions tested before submitting the form.

Additional Project Summary

Team information: U.G.Harshani Sandunika Ranasingha is the individual team lead and sole team member for team HSR. Contact and registration email: harshanisadunika99@gmail.com. WhatsApp: 0727713839, 0769956712.

Project category: AI Agent (primary); related categories: Chatbots and Data Analytics. Domain: AgriTech.

Problem: Farmers may not have timely access to practical, crop-specific guidance that considers their farm conditions and saved observations. General AI answers can be unreliable when they guess at weather, soil measurements, or diagnoses.

Solution: AGRO combines an AI chat assistant with a farmer's crop profiles, saved observations, recorded numeric measurements, and a nearby weather forecast. It retrieves those records when relevant, explains what information it used, and offers cautious next steps without treating estimates as measurements or claiming a certain diagnosis.

Key features: Personalized farm profiles; worldwide farm and crop location selection; crop profiles; Ollama-powered crop chat and crop planning; crop journal; saved measurement trends without estimating missing values; read-only AI data retrieval with visible data-source actions; current weather, forecasts, and historical climate information; local crop-photo review; local document text extraction and questions for PDF, DOCX, TXT, CSV, and Markdown files; local Whisper voice transcription via Transformers.js and FFmpeg; English, Sinhala, and Tamil language preferences; and browser speech playback.

Innovation and responsible use: AGRO connects conversational AI to the farmer's own crop journal and numeric observations, and can retrieve a local forecast as a separate data source. Read-only tools and visible action summaries help distinguish saved measurements and forecast data from AI interpretation. It does not infer soil

nutrients from ordinary photos or present disease suggestions as confirmed diagnoses. Recommendations should be adapted to local crops, climate, and agricultural guidance; AI responses are provisional and do not replace advice from a qualified local agricultural expert.

Technology summary: React 18 and Vite 6 (frontend); Node.js and Express 4 (backend); MongoDB and Mongoose (database); Ollama llama3.2:latest (local text agent and crop planning) and qwen2.5vl:3b (local vision); Transformers.js Whisper Tiny and FFmpeg (local voice transcription); PDF and DOCX text extraction; optional OpenAI API provider; Google Maps Platform APIs; Open-Meteo; JavaScript, HTML, and CSS.

Development status: Working hackathon MVP for local development and testing. Authenticated API smoke tests have verified crop chat and measurement-tool calls, crop planning, crop-photo analysis with a safety disclaimer, PDF and DOCX text uploads, and local voice transcription. On 2026-10-18, all 16 backend tests passed, the frontend production build succeeded, and the backend dependency audit reported zero vulnerabilities. The frontend, backend health endpoint, and Ollama responded with HTTP 200; the configured Ollama text model (llama3.2:latest) and vision model (qwen2.5vl:3b) are installed. This is local-development verification, not a public deployment; no public deployment, repository, screenshot, or demo-video link has been provided.

This supporting summary is prepared from the AGRO project and team details for the NYLP AI Hackathon 2026 submission.

What makes your project innovative? *

AGRO connects conversational AI to the farmer's own crop journal and numeric observations, and can retrieve a local forecast as a separate data source. Its read-only tools and visible action summaries help distinguish saved measurements and forecast data from AI interpretation. It also emphasizes uncertainty: it does not infer soil nutrients from ordinary photos or present disease suggestions as confirmed diagnoses.

Current Project Development Status *

Working hackathon MVP for local development and testing. Authenticated API smoke tests have verified crop chat and measurement-tool calls, crop planning, crop-photo analysis with a safety disclaimer, PDF and DOCX text uploads, and local voice transcription. On 2026-10-18, all 16 backend tests passed, the frontend production build succeeded, and the backend dependency audit reported zero vulnerabilities. The frontend, backend health endpoint, and Ollama responded with HTTP 200; the configured Ollama text model (llama3.2:latest) and vision model (qwen2.5vl:3b) are installed. This is local-development verification, not a public deployment. No public deployment, repository, screenshot, or demo-video link has been provided.

Product & Demo Links

Live Product Link *

NEEDS YOUR INPUT: Deploy the app and enter its accessible URL. It currently has local-development instructions only; do not submit a localhost URL as a public live product.

Demo Type *

NEEDS YOUR INPUT: No public live product or demo video link was provided. If demonstrating locally, select D and provide the demonstration instructions or video link.

- [ ] A — Live product link
- [ ] B — Google Drive video link
- [ ] C — Both live product and video links
- [ ] D — Other demonstration method (describe below)

Project Screenshots Link *

NEEDS YOUR INPUT: Add an accessible Google Drive folder link containing project screenshots.

Please share links rather than uploading large files. For Google Drive resources, set access to Anyone with the link can view or provide the necessary access to the judging panel. Test your links before submitting.

Repository & Technical Documentation

GitHub or Repository Link *

NEEDS YOUR INPUT: This workspace has no public repository URL. Publish the repository or grant the judging panel access, then paste its URL here.

Repository Access Confirmation *

- [ ] I confirm that my repository is publicly accessible or that the judging panel has been granted the necessary access.

README Documentation Checklist *

The README includes the problem and solution, technology stack, setup instructions, AI methodology, model/API details, system workflow chart, tests, and team contribution. Product screenshots still need to be added.

- [x] Problem statement and solution
- [x] Technology stack
- [x] Setup and installation instructions
- [x] AI methodology and implementation
- [x] AI model names and API usage
- [ ] Product screenshots
- [x] Team member contributions

Technology Stack *

React 18 and Vite 6 (frontend); Node.js and Express 4 (backend); MongoDB and Mongoose (database); Ollama llama3.2:latest (local text agent and crop planning) and qwen2.5vl:3b (local vision); Transformers.js Whisper Tiny and FFmpeg (local voice transcription); PDF and DOCX text extraction; optional OpenAI API provider; Google Maps Platform APIs; Open-Meteo; JavaScript, HTML, and CSS.

AI Methodology *

The current local configuration uses Ollama's llama3.2:latest for crop chat, crop planning, and bounded, read-only tools that retrieve saved crop observations, measured values, and local forecast data. Ollama's qwen2.5vl:3b reviews crop photos, and local Whisper Tiny transcribes voice recordings. PDF and DOCX files are converted to text locally before analysis. The assistant uses the farmer's language preference, farm location, crop profile, and saved crop history; it distinguishes recorded facts from inference and never treats a nearby weather forecast as a field measurement. An OpenAI provider is available as an optional configuration.

AI Models and APIs Used *

Configured provider: local Ollama llama3.2:latest for text chat, tools, and recommendations; Ollama qwen2.5vl:3b for image review; Transformers.js onnx-community/whisper-tiny for local speech transcription; and FFmpeg for audio decoding. PDF and DOCX contents are extracted locally before analysis. These local AI features require no OpenAI API key. The backend also retains an optional OpenAI provider (gpt-4o-mini by default) for installations configured to use cloud AI. Open-Meteo provides weather data and is not an AI model.

Additional Documentation Link *

Optional; no additional documentation link has been provided.

Your repository should include a clear README explaining the problem, solution, technology stack, setup steps, AI methodology, model/API usage, screenshots, and team member contributions.

Team Contributions

Individual Team Contributions *

U.G.Harshani Sandunika Ranasingha — individual contributor. The project code includes the React/Vite frontend, Express/MongoDB backend, AI chat-agent integration and data tools, crop/weather/file features, and tests. Please confirm or edit this description so it accurately reflects your personal contribution before submitting.

Development Tools Used *

Visual Studio Code, Node.js/npm, React, Vite, Express, MongoDB, OpenAI API, and Ollama for local model inference.

Challenges Faced and Lessons Learned *

Please personalize: Describe the actual development challenges you encountered and what you learned; this cannot be reliably inferred from the source code.

Final Confirmation

Please confirm the following: *

- [ ] Confirm yourself: I confirm that the information provided is accurate and that my submitted links are accessible to the judging panel.
- [ ] Confirm yourself: I confirm that this project was newly created for the NYLP AI Hackathon 2026 and was not previously launched.
- [ ] I agree to follow the hackathon rules and guidelines.
- [ ] I understand that inaccessible demos, repositories, or incomplete documentation may affect project evaluation.
- [ ] I understand that all projects will be evaluated according to the hackathon's judging criteria.

Additional Information for Judges

AGRO is a local-development hackathon MVP, not a production-ready agricultural diagnostic service. MongoDB is required for persistence. Its local AI uses Ollama text and vision models and locally run speech transcription; the app also offers an optional OpenAI provider when explicitly configured. The first Whisper transcription may download model files from Hugging Face; uploaded audio itself is processed locally. Weather data describes a nearby grid and is not a field measurement. Disease suggestions are not confirmed diagnoses, and farmers should consult a qualified local agricultural officer for serious crop problems. The project was built by extending a React, Express, and MongoDB starter.

Submit

Submission status: Draft — not submitted. Add the missing live demo, screenshots, and repository links; add product screenshots to the README; then review all personal details and final confirmations.

Made with Tally